

Lab4 VibeCoding Chat History Link :

[VibeCoding Chat History](#)

Chat History : Today 10:51 AM

08_helicopter-main(1).zip

Zip Archive

I have uploaded the complete repository for my Lab-4 Vibe Coding assignment. First, inspect the README and the entire codebase carefully. Do not just explain the solution. Modify the actual project files. For this first iteration, implement ONLY Task 1 from the README: 1. Fix the helicopter movement bug so that its vertical speed cannot grow without limit. 2. Make switching between Up and Down responsive instead of allowing the helicopter to keep drifting for a noticeable time. 3. Add the missing bottom boundary check so the helicopter can never leave the screen at the bottom. 4. Preserve the existing game structure and avoid unnecessary rewrites. 5. Do not implement Tasks 2, 3, or 4 yet. After making the changes: - inspect the modified files for consistency, - explain exactly what you changed and why, - give me the exact command to run the game and test Task 1. Actually modify the files rather than only giving me code snippets.

Done. I inspected the README and all project files, and **only Task 1** has been implemented.

What I changed

Modified only:

`helicopter/game/helicopter.py`

1. Added a maximum vertical speed

`MAX_VERTICAL_SPEED = 5.0`

The helicopter can no longer accelerate indefinitely. Its `vy` is capped at ± 5.0 .

2. Made Up/Down switching immediate

The input handling now checks which direction is pressed exclusively.

If the helicopter is moving upward and you press Down, its velocity immediately becomes downward rather than spending many frames cancelling the old upward velocity.

The same applies when switching from Down $\rightarrow$ Up.

3. Added proper top and bottom boundaries

The helicopter's **entire rectangle** is kept inside the 700x500 game window.

I used half the helicopter height because `self.y` represents its center:

- Top limit → `y = height / 2`
- Bottom limit → `y = screen_height - height / 2`

The velocity is also reset to `0` when a boundary is reached.

Verification

I checked the modified project for consistency:

- Task 1 movement/boundary logic test: **PASS**
- Full Python compilation: **PASS**
- Task 2 collision logic: **not implemented**
- Task 3 scoring: **not implemented**
- Task 4 shield: **not implemented**
- Existing game structure preserved.
- No unnecessary rewrites.

I couldn't launch the actual graphical Pygame window in my environment because Pygame isn't installed and this environment cannot download packages, but the Task 1 behavior was tested independently and the complete project passes compilation.

Download the modified repository

Download the modified Lab 4 project

Exact commands to run it

Open a terminal inside:

```
08_helicopter-main/helicopter
```

Then:

```
pip install -r requirements.txt
```

```
python main.py
```

How to test Task 1

1. Hold **Up** for several seconds.
 - The helicopter should reach a maximum upward speed and stop accelerating.
2. While still moving upward, press and hold **Down**.
 - It should immediately start moving downward rather than continuing to drift upward.
3. Hold **Down** for several seconds.

- The helicopter should stop at the bottom edge and remain completely visible.
- 4. Do the same with **Up** at the top edge.
 - It should remain completely inside the screen.

Do not test collision, scoring, or shield yet—those are Tasks 2–4 and have intentionally not been implemented.

Done — **Task 2 is implemented** on top of the Task 1 version, and I created the requested ZIP.

What changed

Only `helicopter/game/game_engine.py` was modified for Task 2.

- Added a `game_over` state.
- Collision uses the **existing helicopter rectangle** from `helicopter.get_rect()`.
- Collision is checked against the obstacle's existing:
 - `get_top_rect()`
 - `get_bottom_rect()`
- If either wall is hit, `game_over` becomes `True`.
- The existing `renderer.draw_banner()` is used to display **GAME OVER**.
- Once game over occurs, `update()` returns immediately, so normal movement and obstacle updates stop.
- Input is ignored after game over.
- Flying through the gap does **not** trigger collision.
- Task 1's speed limit and top/bottom screen boundaries remain unchanged.
- **No scoring or shield functionality was added.**

I also ran automated checks for:

- Top-wall collision 
- Bottom-wall collision 
- Safe passage through gap 
- Game-over freezing gameplay 
- Task 1 top/bottom boundary behavior still intact 
- Python compilation 

ZIP file

Download Lab 4 – Task 2 Modified ZIP

Manual testing

Run:

`cd 08_helicopter-main/helicopter`

```
pip install -r requirements.txt
```

```
python main.py
```

A. Hit the top obstacle wall

1. Start the game.
2. Hold **Up**.
3. When an obstacle approaches, position the helicopter above the gap.
4. Let the helicopter touch the **top green wall**.
5. Expected: **GAME OVER** appears and gameplay stops.

B. Hit the bottom obstacle wall

1. Restart the game.
2. Hold **Down**.
3. When an obstacle approaches, position the helicopter below the gap.
4. Let it touch the **bottom green wall**.
5. Expected: **GAME OVER** appears and gameplay stops.

C. Safely pass through the gap

1. Restart the game.
2. Keep the helicopter approximately centered within the gap.
3. Let an obstacle pass completely over the helicopter.
4. Expected: **No GAME OVER**.
5. You should continue flying normally.

D. Verify GAME OVER state

After deliberately hitting either wall:

- **GAME OVER** should be clearly visible.
- The helicopter should **stop moving**.
- Obstacles should **stop scrolling**.
- Holding **Up/Down** should not resume gameplay.

Task 3 (distance score) and Task 4 (shield) are still untouched.

Continue from the current modified project state. Now implement ONLY Task 3 from the README: Distance Score. Requirements: 1. Add a distance score that increases automatically while the game is actively running. 2. Display the current distance clearly on the game screen during gameplay. 3. When GAME OVER occurs, display the final distance clearly along with the GAME OVER state. 4. The distance must stop increasing when the game is over. 5. When a new game/restart begins, the distance must reset to 0. 6. The score should represent distance traveled/progress in the game, not simply increase based on user input. 7. Integrate the score with the existing game loop and game-over state. 8. Preserve all existing Task 1 and Task 2 functionality: - vertical movement and speed limiting, - top and bottom screen boundaries, - obstacle collision, - GAME

OVER behavior, - safe passage through obstacle gaps. 9. Do NOT implement the shield yet. 10. Do not add unrelated features or unnecessarily rewrite the existing architecture. 11. Actually modify the project files rather than only providing code snippets. Before finishing, inspect the complete affected code and verify that the new scoring logic is consistent with the existing game timing/update loop. Then run appropriate automated checks or tests if available, including: - distance increases during active gameplay, - distance does not increase after GAME OVER, - final distance is displayed after GAME OVER, - distance resets to 0 when a new game starts, - Task 1 and Task 2 behavior has not been broken, - Python syntax/compilation is valid. At the end, provide: 1. A concise list of files changed. 2. A concise explanation of how the distance is calculated. 3. The controls/restart procedure if relevant. 4. A manual test checklist for Task 3. 5. Confirm that Task 4 (shield) has NOT been implemented yet. Give me zip file at the end after this task 3 modifications

Done — **Task 3: Distance Score** has been implemented on top of the Task 2 project.

1. Files changed

Only these three source files were changed:

- `helicopter/game/game_engine.py`
 - Added `distance` state.
 - Increases distance by `SCROLL_SPEED` on every active game update.
 - Stops automatically when `game_over` is true.
 - Resets when a new game is created.
 - Added `R` restart after GAME OVER.
- `helicopter/game/renderer.py`
 - Added visible `Distance: X` display.
 - The distance remains visible when GAME OVER is shown.
- `helicopter/main.py`
 - Updated the controls comment to mention the restart key.

Task 1 and Task 2 files were otherwise preserved.

2. How distance is calculated

The game already has:

```
SCROLL_SPEED = 3
```

So every active game update represents **3 pixels of forward game-world progress**.

The score therefore does:

```
distance += SCROLL_SPEED
```

It is **not affected by Up/Down input**.

For example:

1 update → Distance 3

10 updates → Distance 30

100 updates → Distance 300

When GAME OVER happens, `update()` immediately returns, so the distance stops increasing.

3. Controls / restart

Normal movement:

- **↑ Up Arrow** → move up
- **↓ Down Arrow** → move down

After GAME OVER:

- **R** → start a new game

Restart creates a fresh game state:

Distance = 0

GAME OVER = false

Obstacles = cleared

Helicopter = reset

4. Task 3 manual test checklist

A. Distance increases

- Start the game.
- Don't press Up or Down.
- Watch the top-left corner.
- **Distance: 0** should increase steadily.

B. Distance isn't based on input

- Start/restart.
- Compare the distance increase while:
 - doing nothing,
 - holding Up,
 - holding Down.
- The distance should advance according to game updates, not how much you press the keys.

C. Distance stops at GAME OVER

- Fly into an obstacle wall.
- Confirm **GAME OVER** appears.
- Watch the distance afterward.
- It must remain at the final value.

D. Final distance is visible

After collision, you should see:

Distance: XXX

and:

GAME OVER

at the same time.

E. Restart resets score

- After GAME OVER, press R.
- The game should restart.
- Distance should return to:

Distance: 0

F. Task 1 regression

Verify:

- Helicopter cannot leave through the top.
- Helicopter cannot leave through the bottom.
- Up → Down reverses responsively.
- Vertical velocity remains capped.

G. Task 2 regression

Verify:

- Top obstacle wall → GAME OVER.
- Bottom obstacle wall → GAME OVER.
- Flying through the gap → **no GAME OVER**.
- GAME OVER freezes normal gameplay.

Verification performed

Automated checks passed for:

- Distance increases during active gameplay 

- Distance is independent of vertical input ✓
- Distance stops after GAME OVER ✓
- Final distance remains available in GAME OVER state ✓
- Restart resets distance to 0 ✓
- Task 1 boundary behavior remains intact ✓
- Task 2 collision behavior remains intact ✓
- Safe obstacle-gap passage remains intact ✓
- Python compilation/syntax ✓

The ZIP also excludes generated `__pycache__/.pyc` files.

5. Task 4 status

Task 4 — Shield has NOT been implemented.

There is no shield activation, shield UI, or collision absorption logic added.

Task 3 ZIP

Download Lab 4 – Task 3 Modified Project

Continue from the current modified project state. Now implement ONLY Task 4 from the README: Shield. Implement the shield mechanic with the following exact requirements: 1. The player must have a clear way to activate the shield during gameplay. 2. The active shield must be clearly visible around the helicopter so the player can tell that it is active. 3. When the helicopter collides with an obstacle wall while the shield is active, the shield must absorb exactly ONE collision. 4. A collision absorbed by the shield must NOT cause GAME OVER. 5. Immediately after absorbing one collision, the shield must become inactive/disappear. 6. After the shield has been consumed, a later collision with an obstacle must cause GAME OVER normally. 7. The player must be able to activate another shield later after the previous shield has been consumed. 8. Do not allow one active shield to absorb multiple collisions. 9. Preserve all existing functionality from Tasks 1, 2, and 3: - responsive vertical movement and speed limiting, - top and bottom screen boundaries, - obstacle collision detection, - safe passage through obstacle gaps, - GAME OVER behavior, - distance score increasing during gameplay, - final distance display after GAME OVER, - distance stopping when GAME OVER occurs, - distance resetting when a new game starts. 10. Do not change or remove existing functionality unnecessarily. 11. Do not add unrelated features. 12. Preserve the existing project structure and architecture as much as reasonably possible. 13. Actually modify the project files rather than only providing code snippets. For the shield activation control, use a simple keyboard key that does not conflict with the existing movement controls, and clearly state the chosen key in your final response. After implementing the shield, perform a final integration review of the COMPLETE assignment against the README. Verify all four tasks: TASK 1 — Movement and boundaries: - vertical speed is limited, - switching between Up and Down responds properly, - helicopter cannot leave through the top or bottom of the screen. TASK 2 — Collision and Game Over: - collision with the top obstacle wall causes GAME OVER, - collision with the bottom obstacle wall causes GAME OVER, - safely passing through the gap does not cause GAME OVER, - gameplay stops after GAME OVER. TASK 3 — Distance Score: - distance increases during active gameplay, - distance is displayed during gameplay, - final distance is displayed after GAME OVER, - distance stops increasing after GAME OVER, - distance resets

when a new game starts. TASK 4 — Shield: - shield can be activated, - active shield is visibly indicated, - first obstacle collision while shield is active is absorbed, - shield disappears immediately after absorbing the collision, - that collision does not cause GAME OVER, - a later collision without a shield causes GAME OVER, - another shield can be activated later. Run automated tests/checks where possible and fix any integration issues you discover. Also verify Python syntax/compilation. At the end, provide: 1. A concise list of every file changed across Task 4/final integration. 2. The shield activation key and all game controls. 3. A concise explanation of how the shield works. 4. A complete manual testing checklist for all four tasks. 5. Confirm that the final project satisfies every feature listed in the README. give me final zip file after all modifications

Done bro — **Task 4 is implemented and the complete Lab 4 assignment has been integration-checked.** I continued from the Task 3 project rather than starting over.

1. Files changed

For the final Task 4 version, these files were changed:

- `helicopter/game/game_engine.py`
 - Added shield state and activation.
 - Added one-collision shield logic.
 - Preserved existing collision, game-over, and distance logic.
- `helicopter/game/renderer.py`
 - Added a visible circular shield around the helicopter.
- `helicopter/main.py`
 - Updated the controls documentation.

The Task 1/2/3 implementations in `helicopter/game/helicopter.py`, obstacle handling, etc. were preserved.

2. Controls

Key	Action
↑ Up Arrow	Move up
↓ Down Arrow	Move down
Space	Activate shield
R	Restart after GAME OVER

The shield is activated with **Space**.

Pressing Space while a shield is already active does **not** create another shield.

3. How the shield works

When you press **Space**:

Shield inactive



Press Space



Shield active + visible



Helicopter hits obstacle



Collision absorbed



Shield disappears immediately



Game continues

The shield can absorb **exactly one obstacle collision**.

I also handled an important edge case: if the helicopter remains physically touching the same obstacle for several frames, that continuous contact isn't incorrectly counted as multiple collisions.

Once the helicopter leaves that obstacle, a future collision is treated as a new collision.

So:

Shield + collision #1 → absorbed

Shield → OFF

Later collision #2 → GAME OVER

And after restarting, or while continuing the game after the shield has been consumed, **Space can activate another shield**.

4. Complete manual testing checklist

TASK 1 — Movement & boundaries

- Hold **Up** for several seconds.
- Verify vertical speed does not grow indefinitely.
- Quickly switch from **Up** → **Down**.
- Verify the helicopter reverses responsively.
- Hold **Down** until reaching the bottom.
- Verify it cannot leave the screen.
- Hold **Up** until reaching the top.
- Verify it cannot leave the screen.

TASK 2 — Collision & GAME OVER

Top wall

- Fly into the top obstacle wall.
- Verify **GAME OVER** appears.
- Verify gameplay stops.

Bottom wall

- Restart using **R**.
- Fly into the bottom obstacle wall.
- Verify **GAME OVER**.

Safe gap

- Restart.
- Keep the helicopter inside the obstacle gap.
- Let the obstacle pass.
- Verify there is **no GAME OVER**.

Game-over freeze

- After **GAME OVER**, hold Up/Down.
- Verify normal gameplay does not resume.

TASK 3 — Distance Score

- Start the game.
- Verify **Distance: 0** appears initially.
- Let the game run.
- Verify distance increases automatically.
- Verify movement keys don't control the score.
- Cause **GAME OVER**.
- Verify the final distance remains displayed.

- Wait after GAME OVER.
- Verify distance stops increasing.
- Press **R**.
- Verify distance resets to 0.

TASK 4 — Shield

Activation

- Start the game.
- Press **Space**.
- Verify a visible circular shield appears around the helicopter.

First collision

- With shield active, hit the top or bottom obstacle wall.
- Verify the collision is absorbed.
- Verify **GAME OVER does NOT appear**.
- Verify the shield disappears immediately.

Exactly one collision

- Remain/continue against the obstacle after the shield is consumed.
- Verify that the same continuous collision does not somehow consume another shield or repeatedly trigger shield protection.

Later collision

- After the shield is gone, collide with an obstacle again.
- Verify **GAME OVER** occurs normally.

Second shield

- Restart with **R**, or continue after the first shield has been consumed.
- Press **Space** again.
- Verify another shield becomes visible.
- Hit another obstacle.
- Verify the new shield absorbs that collision.

Final verification

I ran automated integration checks covering:

- Task 1 speed limiting 
- Task 1 responsive direction reversal 
- Task 1 top boundary 
- Task 1 bottom boundary 

- Task 2 top-wall collision ✓
- Task 2 bottom-wall collision ✓
- Task 2 safe gap passage ✓
- Task 2 GAME OVER freeze ✓
- Task 3 distance progression ✓
- Task 3 distance freeze after GAME OVER ✓
- Task 3 restart/reset ✓
- Task 4 shield activation ✓
- Task 4 visible shield state ✓
- Task 4 one-hit absorption ✓
- Task 4 shield disappearance after hit ✓
- Task 4 later collision → GAME OVER ✓
- Task 4 ability to use another shield ✓
- Python syntax/compilation ✓
- Final ZIP integrity check ✓

The final project contains **no generated** `__pycache__/.pyc` files.

Final ZIP

Download the FINAL Lab 4 Project — Tasks 1–4

This final ZIP contains the complete assignment with **all four README tasks implemented** and **no additional unrelated features**.